

UCab — Full-Stack MERN Cab Booking Platform

Solo Developer / Team Member: Shaik Sumiya Zainab | Project ID: N/A (Solo Track)

Phase 2: Requirement Analysis — Technology Stack Specification

Project Name: Cab Booking (`UCab`)

Project ID: `N/A (Solo Track Submission)`

Team Member / Solo Developer: Shaik Sumiya Zainab

1. Stack Selection Rationale

Developing a full-stack, enterprise-grade cab booking platform as a solo developer requires an engineering stack that maximizes code reuse, type/syntax familiarity across front and back ends, rapid prototyping capabilities, and seamless deployment. The MERN Stack (MongoDB, Express.js, React.js, Node.js) was selected because JavaScript serves as the universal language across all application layers, eliminating context switching and enabling rapid end-to-end feature delivery.

2. Comprehensive Technology Layer Breakdown

2.1 Presentation & Client Layer (Frontend)

- Core Library: `React.js 18` — Selected for its component-based virtual DOM architecture, functional hooks (`useState`, `useEffect`, `useContext`), and state predictability.
- Build Tool & Bundler: `Vite 5+` — Provides ultra-fast Hot Module Replacement (`HMR`) and highly optimized production builds compared to traditional Webpack setups.
- Routing: `React Router DOM v6` — Handles declarative client-side routing across 11 distinct user and admin screens with role-protected route wrappers (`ProtectedUserRoute.jsx` and `ProtectedAdminRoute.jsx`).
- HTTP Client: `Axios` — Intercepts and sends REST API requests with automatically injected `Authorization: Bearer <token>` headers.
- UI/UX & Iconography: `Lucide React Icons` & `Vanilla CSS 3 (index.css)` — A tailored, dependency-free design system utilizing CSS variables, glassmorphism (`backdrop-filter: blur(10px)`), and responsive grid layouts without CSS framework bloat.

2.2 Application Server Layer (Backend API)

- Runtime Environment: `Node.js v20+` — Non-blocking, event-driven I/O model ideal for handling concurrent REST API requests from multiple active cab riders and dispatchers.
- Web Framework: `Express.js v4` — Lightweight HTTP routing framework used to construct structured REST endpoints (`/api/users`, `/api/cars`, `/api/bookings`, `/api/admin`).
- Security & Authentication:
 - `JSON Web Token (jsonwebtoken)` — Stateless, cryptographically signed Bearer tokens ensuring tamper-proof user and admin session authentication.
 - `Bcrypt.js` — Industry-standard password hashing algorithm with configurable salt rounds protecting credentials at rest.
- Middleware & File Uploads:
 - `Multer` — Multipart/form-data handling for processing cab fleet image uploads.
 - `CORS` — Cross-Origin Resource Sharing configuration enabling clean client-server communication across localhost ports (`5173/5174` != `8000`).

2.3 Data Persistence & Resilience Layer (Database)

- Primary ORM: `Mongoose v8` — Object Data Modeling (`ODM`) library providing schema enforcement (`User.js`, `Car.js`, `Booking.js`, `Admin.js`), validation, and query builders for MongoDB.
- Zero-Config Fallback Storage Engine (`server/db/store.js`): To solve the critical challenge of database crashes during live university presentations or isolated grader testing, an atomic JSON persistence engine was custom-built using Node's `fs` module. When `global.isMongoConnected` evaluates to false, all controller

methods automatically switch from Mongoose queries to synchronous atomic file read/write operations against `server/db/data.json`.

3. Version Audit & Dependency Summary Table

Package / Tool	Exact Version	Spec	Primary Role in UCab
react / react-dom	^18.3.1		UI presentation, virtual DOM rendering, and custom state context (<code>AuthContext</code>)
vite	^5.4.1		Next-generation frontend tooling and local dev server
react-router-dom	^6.26.1		Single-page application navigation and role-based route gating
lucide-react	^0.436.0		High-fidelity vector iconography across user widgets and admin dashboards
express	^4.19.2		Backend REST API server and routing pipeline
mongoose	^8.5.3		Schema definition and data validation for MongoDB
jsonwebtoken	^9.0.2		Cryptographic session signing (<code>JWT_SECRET</code>)
bcryptjs	^2.4.3		Password encryption for rider and admin credentials
multer	^1.4.5-lts.1		Handling local and memory-based vehicle image attachments